

Brain MRI Tumor Segmentation and Quantitative Analysis Using Classical Image Processing

Week 3 – Basic Filtering TryON and Finalizing Methodsethology

Kush Kelaiya

Enrollment No.: AU2440018

*School of Engineering and Applied Science
Ahmedabad University
Ahmedabad, Gujarat, India*

Rushin Modi

Enrollment No.: AU2440028

*School of Engineering and Applied Science
Ahmedabad University
Ahmedabad, Gujarat, India*

Kritee Shah

Enrollment No.: AU2440080

*School of Engineering and Applied Science
Ahmedabad University
Ahmedabad, Gujarat, India*

Jash Odedara

Enrollment No.: AU2440096

*School of Engineering and Applied Science
Ahmedabad University
Ahmedabad, Gujarat, India*

Submitted By:- G9 - The Medics.

I. WORK DONE THIS WEEK:-

This week, the project was extended from the initial visual exploration of brain MRI scans toward a systematic classical image-processing approach for identifying abnormal regions. The main idea is to establish a *normal baseline* using brain MRI scans without tumors and then compare a test scan against this baseline. The difference image is used to locate regions whose intensity behaviour is sufficiently different from the normal variation observed in the baseline scans.

The earlier work established the basic understanding of the MRI data, including opening `.nii.gz` files in Python, visualising scans, manually observing possible tumor boundaries, and studying relevant segmentation literature. The previous report also identified the need for systematic preprocessing, tumor localisation, segmentation, and quantitative analysis. These objectives are retained in the current approach.

The current methodology therefore focuses on four connected stages: (i) constructing a baseline from multiple tumor-free scans, (ii) characterising and rejecting normal differences, (iii) detecting and marking the remaining abnormal regions, and (iv) accelerating the computationally expensive multidimensional image operations using CUDA and GPU processing.

A. Baseline MRI Dataset

Instead of treating a single normal MRI scan as the reference, the current approach uses multiple MRI scans without tumors. Five normal brain MRI scans are considered as the initial baseline set. Using multiple scans is important because a subtraction between only two scans can contain differences

caused by normal anatomical variation, intensity variation, acquisition conditions, or other non-tumor effects.

For two baseline volumes M_1 and M_2 , the absolute voxel-wise residual is defined as

$$D_{12}(x, y, z) = |M_1(x, y, z) - M_2(x, y, z)|. \quad (1)$$

This operation is repeated for the available baseline scans and across the relevant slices/axes. The resulting difference volumes are used to estimate the range of differences that can occur even when both scans are considered normal.

B. Why Multiple Baseline Images Are Required

A single reference image can incorrectly classify ordinary anatomical or acquisition-related differences as abnormalities. The use of several tumor-free scans provides a sample of normal variation. Instead of asking whether a test voxel is simply different from one reference scan, the method asks whether its difference is outside the variation observed across the normal baseline.

For a baseline set $\{M_1, M_2, \dots, M_N\}$, a baseline reference volume can be represented using a voxel-wise statistic such as the median:

$$B(x, y, z) = \text{median}_{i=1, \dots, N} M_i(x, y, z). \quad (2)$$

A test volume T can then be compared with the baseline as

$$D_T(x, y, z) = |T(x, y, z) - B(x, y, z)|. \quad (3)$$

The exact aggregation statistic can be changed during experimentation. The median is useful as a robust reference because a single unusually high or low voxel value in one baseline scan has less influence on the reference than it would have under a simple arithmetic mean.

C. Establishing and Ruling Out Normal Differences

An important part of the proposed segmentation method is not to interpret every difference as a tumor. Normal MRI scans themselves produce non-zero residuals after subtraction. Therefore, the baseline residuals are used to establish a threshold for abnormality.

The baseline differences give us the information about the normal differences. Across multiple baseline comparisons. We can identify these baselines and rule out when the test cases are used. The remaining part, which will be above threshold will give us a result for a possible brain tumor. When checked with multiple baselines, this will provide us with a true positive or a true negative. This method also helps avoid false positives and false negatives.

D. Abnormal Region Detection and Marking

Once the normal-difference threshold has been established, we can identify the outliers/abnormal parts from the threshold. Our filters and masks are the brain MRI scans which do not have tumors in them. The objective is to mark the actual different part of the brain volume rather than merely reporting that a scan is different. The marked region can then be visualised on top of the original MRI slices. Doing this across multiple established baselines will provide more concrete results for the MRI.

E. Quantitative Analysis of the Detected Region

After the abnormal region has been segmented, its statistics can be calculated. The current analysis is intended to report at least the following quantities:

- Number of detected voxels.
- Estimated physical volume using the voxel spacing stored in the NIfTI metadata.
- Bounding-box dimensions of the detected region.
- Centroid/location of the detected region.

When the tumor voxels are counted after identifying them, we can use the voxel data to get the approximate volume of the tumor as follows

$$V = N_v C_t. \quad (4)$$

N_v = Voxel volume

C_t = Count of tumor voxels.

This provides a quantitative output in addition to the visual segmentation.

II. GPU-ACCELERATED IMAGE PROCESSING

The MRI volumes are multidimensional data structures, and operations such as subtraction, absolute difference, thresholding, masking, and statistical calculations can involve a large number of voxel-wise operations. The project therefore explores GPU acceleration for these operations.

The hardware notes used for the current implementation identify an NVIDIA GPU with 4+ GB of VRAM and 2048+ CUDA cores as the available/reference configuration. The system RAM requirement considered in the current scenario

TABLE I
HARDWARE CONFIGURATION USED FOR THE IMPLEMENTATION.

Component	Configuration
GPU	NVIDIA RTX 4070 Laptop GPU, [8gb VRAM], [4608 CUDA cores]
CPU	Intel Core i7 13650HX
System RAM	32gb RAM
CUDA version	CUDA 13.4

is approximately 8–32 GB. The exact hardware configuration used for the final experiments should be recorded in the table below.

A. Python and CUDA Experiments

The initial GPU implementation was explored from Python. The image-processing pipeline uses Python for loading, checking, visualising, and analysing MRI data, while CUDA-enabled libraries are used for GPU computation.

The libraries explored for CUDA-based processing include: `leftmargin=*`

- `cupy-cuda13x`
- `cupy-cuda13x[ctk]`
- `cuda-python`

The intended processing path is to transfer suitable 3D MRI arrays to GPU memory and perform multidimensional operations there rather than repeatedly processing individual voxels through Python-level loops.

B. Limitations of the Python-Based GPU Approach

The Python implementation demonstrated that GPU acceleration can be useful, but it also exposed limitations in the current implementation. MRI volumes contain a large amount of 3D image data, and maintaining multiple volumes, intermediate arrays, difference maps, masks, and statistical buffers can consume substantial VRAM.

In addition, GPU acceleration does not automatically make every part of a Python program fast. Operations that remain inside CPU-side Python loops can become a bottleneck. Repeated host-to-device and device-to-host memory transfers can also reduce the benefit of GPU computation. Therefore, the current implementation needs to minimise Python-level loops and keep suitable bulk operations on the GPU.

These observations motivate the transition from a Python-heavy implementation toward a C/C++ implementation for the computational core.

III. TRANSITION TO C/C++ FOR THE COMPUTATIONAL CORE

The current approach modifies the original Python-heavy processing pipeline by moving the performance-critical image-processing operations toward C/C++. The objective is not to remove Python completely, but to separate the computational core from the visualisation and verification layer.

The intended division is:

`leftmargin=*`

- **C/C++ and CUDA:** loading/handling prepared image data where appropriate, multidimensional matrix operations, subtraction, thresholding, masking, and other computationally expensive operations.
- **Python:** visual checks, result verification, plotting, slice selection, inspection of intermediate outputs, and comparison of the C/C++ results with expected image behaviour.

This division allows the project to retain Python's convenience for visualisation and analysis while using a lower-level language for the repeated numerical operations that dominate processing time.

IV. CURRENT PROCESSING WORKFLOW

The complete current methodology can be summarised as follows:

- 1) Load multiple tumor-free brain MRI scans.
- 2) Convert the scans into a common representation suitable for voxel-wise comparison.
- 3) Perform the required preprocessing and spatial consistency checks before subtraction.
- 4) Calculate pairwise differences between normal scans.
- 5) Use the normal difference distribution to identify and rule out expected differences.
- 6) Establish an abnormality threshold from the normal baseline.
- 7) Compare the test MRI volume with the baseline reference.
- 8) Generate a thresholded abnormality mask.
- 9) Remove or suppress irrelevant isolated differences using classical image-processing operations.
- 10) Mark the remaining abnormal region on the original MRI slices.
- 11) Calculate quantitative statistics for the detected region.
- 12) Perform computationally expensive operations using C/C++ and CUDA where beneficial.
- 13) Use Python for visual verification, slice inspection, plotting, and validation of the generated results.

V. WORK TO BE DONE NEXT WEEK

- Finalise the preprocessing required before voxel-wise subtraction.
- Implement the baseline residual calculation for all five normal MRI scans.
- Evaluate suitable thresholding strategies using the observed normal-difference distribution.
- Implement the abnormal-region mask and classical post-processing.
- Produce slice-wise overlays of the detected region.
- Implement the computationally intensive path in C/C++ with CUDA.
- Calculate volume and other statistics for the detected region.
- Validate the output against available annotated data where applicable.

VI. DATASET AND PROJECT CONTEXT

The project continues to use the OpenNeuro dataset referenced in the previous report. The earlier work also reviewed research on frequency-domain methods and an annotated brain tumor dataset to understand segmentation and ground-truth annotations. These sources are being used as background and validation references while the current implementation remains focused on classical image-processing methods.

Dataset: <https://openneuro.org/datasets/ds005003/versions/2.0.0>

VII. CONCLUSION

The current work develops the project from initial MRI inspection toward a measurable baseline-subtraction methodology. Multiple tumor-free scans are used to estimate normal differences, a threshold is established to suppress expected variation, and the remaining abnormal region is marked and quantified. GPU acceleration is being introduced because the processing involves large multidimensional MRI volumes and repeated matrix operations.

The experiments with Python and CUDA also showed that GPU acceleration must be considered together with memory usage and program structure. Large 3D arrays can consume substantial VRAM, while CPU-side loops can limit performance. The next implementation therefore moves the performance-critical processing toward C/C++ and CUDA while retaining Python for visual verification, slicing, plotting, and result analysis.

Repo Link:-

https://github.com/Rushin-Mod/DIP_2026_P20_G9

VIII. REFERENCES

REFERENCES

- [1] M. Shao et al., "Rethinking Brain Tumor Segmentation From the Frequency Domain Perspective," in *IEEE Transactions on Medical Imaging*, vol. 44, no. 11, pp. 4536-4553, Nov. 2025, doi: 10.1109/TMI.2025.3579213.
- [2] A. Fateh et al., "BRISC: Annotated dataset for brain tumor segmentation and classification," *Nature News*, <https://www.nature.com/articles/s41597-026-06753-y> (accessed Sep. 19, 2026).
- [3] M. Kalyan and A. K. Sharma, "Comparative Study of Classical Image Processing Techniques for Brain Tumor Segmentation in MRI Scans," *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, vol. 13, no. VI, Jun. 2025, doi: 10.22214/ijraset.2025.72609.
- [4] "OpenNeuro," Openneuro.org, 2026. <https://openneuro.org/datasets/ds005003/versions/2.0.0> (accessed Sept. 12, 2026).